

1) Define Asymptotic notation & explain its importance in analyzing algorithm efficiency.

A.) Asymptotic notation is a mathematical way to describe the growth rate of an algorithm's running time or space requirement as the input size (n) increases.

- It helps to express the performance of an algorithm independent of machine.

Common Asymptotic Notations:

- Big $O(O)$ - gives the upper bound (worst-case)
- Omega (Ω) - gives the lower bound (best-case)
- Theta (Θ) - gives the tight bound (aver. or exact case)

Importance:

- Helps compare algorithm's efficiency without running them.
- Focuses on scalability (how performance changes as input grows).
- Removes dependency on hardware, compiler or any programming language.

300 6500
190

490
4920

2) Explain the difference between tail & head recursion with examples.

Sol: Recursion is the process ^{when a} ~~calling~~ function calls itself to solve smaller parts of a problem.

- len,
Based on where recursive call appears in the function, it can be tail or head recursion.

1) Tail recursion:

→ The recursive call is the last statement in the function.

→ No pending operations after the recursive call.

→ can be optimized by the compiler to save memory.

Example: void tailRec(int n) {

if (n == 0) return;

cout << n;

tailRec(n-1); // recursive call at the end.

}

2) Head recursion:

→ The recursive call occurs before any other operation in the function.

→ Operations are performed after returning from the recursive call.
→ Can't be optimized like tail recursion.

Ex: void headRec(int n) {

if (n == 0)

return;

headRec(n-1); // recursive call first

cout << n;

}

3) Derive the index formula for accessing element in a 2D array stored in row-major order.

A.)

1) Let the 2D array have R rows & C columns.

2) Elements are stored in row-major order.

3) B → Base address

4) We use zero based indexing.

5) In row-major, every full row before row 'i' contributes C elements each.

So no. of elements that come before element (i, j) is

$$= i \times C + j$$

each element occupies S bytes.

$$\Rightarrow (i \times C + j) \times S$$

∴ address of element (i, j) is

$$\boxed{\text{base} + (i \times C + j) \times S}$$

If indexing starts from 1 of $1 \leq i \leq R, 1 \leq j \leq C$

$$\text{address}(i, j) = \text{base} + ((i-1) \times C + (j-1)) \times S$$

4) Explain the difference between linear and bin-ary search with their time complexities

A) Linear search:

- Checks each element one by one until target is found.
- Works on unsorted or sorted data.
- Element found at first position → $O(1)$ Best case
- Element not found or last element → $O(n)$ Worst case
- Efficiency ⇒ slower for larger datasets.

Binary search:

- Repeatedly divides the sorted array in half to locate the target
- works only on sorted data
- Middle element is the target ⇒ $O(1)$ Best case
- Requires multiple divisions ⇒ $O(\log n)$ Worst case

→ efficiency ⇒ much faster for larger sorted datasets.

5) Write the algorithm and explain the working of insertion sort with an example.

A) Algorithm:

- Start from the 2nd element (index=1)
- compare it with elements before it.
- shift all elements greater than the key to one position ahead.
- Insert the key in its correct position.
- repeat for all elements until the array is sorted.

Ex: Array → [5, 3, 4, 1]

- 1) key = 3 → compare with 5 → shift 5 → [3, 5, 4, 1]
- 2) key = 4 → comp. with 5 → shift 5 → [3, 4, 5, 1]
- 3) key = 1 → compare with 3, 4, 5 → shift all.
sorted array = [1, 3, 4, 5]

Time complexity ⇒ Best cases: $O(n)$ already sorted
Worst case: $O(n^2)$ (reverse order)

6) Discuss how sparse matrices are represented & explain any one representation method.

A) Sparse matrix $\rightarrow$ a matrix that has mostly zero elements and only a few non-zero values.

Storing all elements wastes memory, so we use special representations to save space.

Common representation methods:

1) 3-tuple (triplet) representation.

2) Linked list representation.

In this method, only non-zero elements are stored in a table with three columns:

Row	Column	Value

represents the index of the row where the non-zero element is present.

particular value present

represents the index of the column where the non-zero element is present

Eg:

0	1	2	3	4
0	0	4	0	5
0	0	0	3	6
0	0	0	2	0
0	2	3	0	0

Row	0	1	2	3	4
Column	2	4	3	4	3
Value	4	5	3	6	2

8) a) Write a recursive algorithm for the tower of Hanoi problem & explain it.

A) Algorithm:

- 1) If only 1 disk, move it directly from A $\rightarrow$ C
- 2) Otherwise:

- Move $n-1$ disks from A $\rightarrow$ B using C as auxiliary.

- Move the n th (largest) disk from A $\rightarrow$ C

- Move the $n-1$ disks from B $\rightarrow$ C using A as auxiliary.

Code:

Public class ~~Towers~~ TowersOfHanoi

Static void towersOfHanoi(int n, char source, char

Auxiliary, char destination)

{ if (n == 1) {

cout << "move disk 1 from " + source +

"to" + destination);

return;

}

// Step 1 - move $n-1$ disks from source to auxiliary

towersOfHanoi(n-1, source, destination, auxiliary);

// Step 2 - move the largest disk to destination.

cout << "move disk " + n + " from " + source + "to" + destination;

11 step: 3 - move $n-1$ disks from auxiliary to destination

~~12~~ towerOfHanoi($n-1$, auxiliary, source, destination);

} public static void main (String[] args) {

int $n = 3$; // no. of disks

towerOfHanoi(n , 'A', 'B', 'C');

}
}
Time complexity:

$$T(n) = 2T(n-1) + 1 \rightarrow O(2^n).$$

Space complexity:

$$O(n).$$

Set-2

1) Time-space tradeoff: using more memory to reduce the execution time, or using less memory at the cost of more time.

ex: fast more memory: store factorials in an array $\rightarrow O(1)$, $O(n)$ space

slow, less memory: compute factorial each time $\rightarrow O(n)$ time, $O(1)$ space

2) Multidimensional arrays are used to store data in the a table like or grid structure (rows & columns)

Applications:

1) Matrices in maths: used for matrix operation like addition, subtraction & multiplication

ex: `int [][] matrix = {{1,2,3},{3,4,5}};`

2) Image processing: each pixel color values (RGB) are stored in a 2D array.

3) In Game boards like chess, tic, tac, toe etc...

3) For a 1D array

Address of element (i) = Base Address (B) + (i - LB) × size

where,

B → Base address of 1st element

i → Index of the desired element.

LB → Lower bound.

Size → memory size of each element.
(in bytes)

Significance:

→ Helps the computer directly access any element without looping.

→ Provides fast random access.

→ Time complexity $O(1)$.

4) Linear search:

→ Start from the first element of the array.

→ Compare the current element to the key
{key → the element to be found}

→ If they are equal, return the position/index of the element.

→ If not move to the next element.

→ Repeat this process till it reaches the end of array.

→ If the element is not found, return -1.

Pseudocode

Linearsearch(arr[], n, key)

for ($i=0$; $i < n$; $i++$) {

if ($arr[i] == key$) {

return i

}

return -1

}

Advantages:

→ Simple & easy to implement.

→ Works on unsorted arrays.

Limitations:

→ Doesn't work on large scale applications.

→ Slow if element is at the end.

Time complexity:

Best $O(1)$

Avg $O(n/2)$

Worst $O(n)$

5) Quick sort:

1) Picking a pivot element.

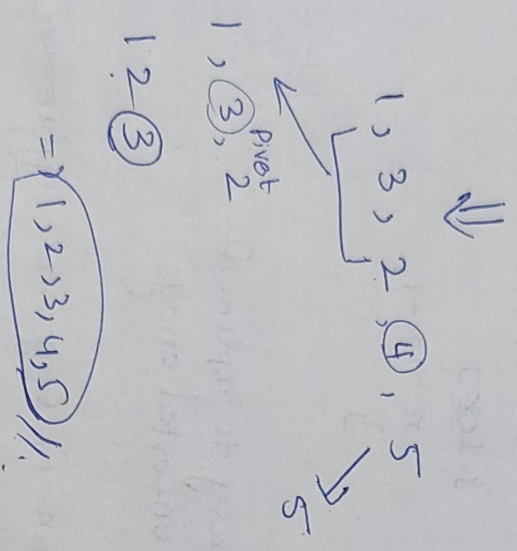
2) Partitioning the array such that

→ all the elements smaller than pivot are on left side

→ elements greater than pivot → right side
 → pivot is now in its correct position.
 → Recursion calls on the subarrays to sort them.

→ Stop recursion when subarray size is 0 or 1.

ex: 5, 4, 3, 2, 1
 let pivot = 4



6) Recursion: The process of a function calling itself is known as recursion.

1) Direct recursion → function calls itself directly

ex:

```
int factorial(int n) {
    if (n == 0) return 1;
    return n * factorial(n-1);
}
```

2) Indirect Recursion:

2) Tail recursion: Recursion call is the last statement in the fn.

ex: sum of 1st n numbers

```
int sum(int n, int s) {
    if (n == 0) return s;
    return sum(n-1, s+n);
}
```

3) Non-tail recursion: Recursion call is not the last statement. Some computation happens after returning.

ex: factorial.

```
int factorial(int n) {
    if (n == 0) return 1;
    return n * factorial(n-1);
}
```

8) a) Public class Fibonacci:

public static void main(String[] args) {

```
    int n = 10;
    int a = 0;
    int b = 1;
    int c;
```



```
cout (<" + b + ">);
```

```
for (int = 0; i <= n; i++) {
```

```
    int c = a + b;
```

```
    cout (<" + ">);
```

```
    a = b;
```

```
    b = c;
```

using
Recursion:

```
int fib(int n) {
```

```
    if (n <= 1) return n;
```

```
    return fib(n-1) + fib(n-2);
```

b) removing recursion: recursion can be replaced by iteration (loops) using simple variables and loops.

ex: recursive version:

```
int fib(int n) {
```

```
    if (n <= 1) return n;
```

```
    return fib(n-1) + fib(n-2);
```

converted to iterative:

```
int fib(int n) {
```

```
    int a = 0, b = 1, c;
```

```
    for (int i = 0; i <= n; i++) {
```

```
        c = a + b
```

```
        a = b;
```

```
        b = c;
```

```
    } return b;
```

Set 3

1) Algorithm efficiency means how fast & how little memory an algorithm uses to solve a problem.

types are

↳ Time efficiency

↳ space efficiency.

↳ how much time it takes to run.

↳ how much memory it uses.

It is measured using Big O notation ($O()$), it shows the growth rate of time or space with

Input size n .

ex: $O(1) \rightarrow$ constant & fastest

$O(n) \rightarrow$ linear & medium

$O(n^2) \rightarrow$ Quadratic & Slow

2) Time complexity:

1) Measure how much time it takes for an algo to run.

2) Depends on no. of steps executed

3) Goal is to make the Program run faster

ex: $O(n)$ time for a loop running n times

3) Let the 3-D array be

$A[L_1, U_1, L_2, \dots, U_2, \dots, L_i, \dots, U_i, \dots]$

$L_i \rightarrow$ lower bound

$U_i \rightarrow$ upper bound

Space complexity

1) It measures how much memory an algo uses

2) Depends on amount of variables.

3) Goal is to make ~~program~~ use the less memory

ex: $O(n)$ space $\rightarrow$ storing n elements in an array.

for element $A[i][j][k]$ address $(A[i][j][k])$

~~= Base Address + ((i - lb_i) * np + (j - lb_j) * p + (k - lb_k)) * size~~

~~Base Address~~

$B.A + [(i - lb_i) \times np + (j - lb_j) \times p + (k - lb_k)] \times$
size

4) Binary search is used to find an element in a sorted array by dividing array into half.

Steps:

1) Initialize $(n = \text{size})$.

low = 0

high = $n - 1$

2) Find mid

mid = $(\text{low} + (\text{high} - \text{low}) / 2)$

3) compare

if $(\text{mid} == \text{key}) \rightarrow$ element found

if $(\text{mid} < \text{key}) \rightarrow \text{low} = \text{mid} + 1$

if $(\text{mid} > \text{key}) \rightarrow \text{high} = \text{mid} - 1$

4) Repeat steps until the element is found

5) Bubble sorting is the simplest sorting algorithm - it works by repeatedly swapping adjacent elements if they are in the wrong order.

Steps:

- 1) start with the 1st element of the array
- 2) Compare the current element with the next one
- 3) swap them if they are in wrong order.
- 4) move to the next pair and repeat the comparison
- 5) After the 1st pass the ~~last~~ largest element comes to end.

6) repeat the passes for remaining elements

ex: [5, 2, 4, 1, 3]

Pass-1:

5 2 4 1 3

2 5 4 1 3

2 4 5 1 3

2 4 1 5 3

2 4 1 3 5 //

Pass-2:

2 4 1 3 5

2 1 4 3 5

2 1 3 4 5 //

Pass-3:

2 1 3 4 5

1 2 3 4 5 //

6) int factorial(int n) {

if (n == 0 || n == 1) return 1;

else

return n * factorial(n-1);

↙ 3

recursive call

factorial of n = n(n-1)(n-2).....

ex: for n=4

step	function call	return value
1	factorial(4)	4 * factorial(3)
2	factorial(3)	3 * factorial(2)
3	factorial(2)	2 * factorial(1)
4	factorial(1)	base case reached

now it unwinds

factorial(2) = 2 * 1 = 2

" (3) = 3 * 2 = 6

" (4) = 4 * 6 = 24 //

3) a) Recursion

- Function calls itself repeatedly until
- base condition is reached.
- uses more memory, coz each function call is stored in stack.
- It is slower.
- Needs base condition to stop recursion.

b) Iteration:

- uses loops (like for, while) to repeat steps.
- uses less memory, since variables are reused.
- It is faster.
- Stops when loop condition fails.

b) algorithm:

```
Fibonacci(n)
if (n == 0)
    return 0
else if (n == 1)
    return 1
```

else
return fibonacci(n-1) + fibonacci(n-2)

Set 4:

1) algorithm: step by step procedure to solve a problem or perform task.

Efficiency:

- saves time
- saves memory
- handle big data

2) Row-major order:

- stores array row by row.
- All elements of 1st row are stored first, then the second row & so on.

ex: $A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$

⇒ 1, 2, 3, 4, 5, 6 //

Column major order

- Stores the array column by column.
- All elements of 1st column are stored first, then the 2nd column, & so on.

ex: $[11, 2, 3, 4, 5, 6]$

$\Rightarrow 1, 4, 2, 5, 3, 6 //$

3) Multidimensional array:

Address $(A[i][j][k])$

$\Rightarrow$ ~~Base +~~ ~~$(i-1) \times (n-l_2+1) + (j-1) \times (n-l_3+1) + (k-1)$~~

Base + $[(i-1) \times (n-l_2+1) + (j-1) \times (n-l_3+1) + (k-1)]$

4) Insertion sort: Sorting the array by inserting each element into its correct position

Steps:

- 1) start with 2nd element
- 2) compare it with the element before it.
- 3) ~~shift~~ shift larger elements to right
- 4) Insert the current element in the correct position.

Ex: Arr = $[5, 3, 4]$

$[3, 5, 4]$

$[3, 4, 5] //$

Time complexity:

best: $O(n)$

worst: $O(n^2)$

Average: $O(n^2)$

Selection sort: The smallest element in the unsorted part is placed at the beginning

Steps:

- 1) Find the smallest element ~~sort~~
- 2) swap it with the 1st element
- 3) Repeat for the remaining unsorted part

ex: $[5, 3, 4]$

$\hookrightarrow [3, 5, 4] \rightarrow [3, 4, 5]$

Time complexity:

best: $O(n^2)$

worst: $O(n^2)$

Average: $O(n^2)$

6) Removing recursion: (converting to Iteration)
to remove recursion, we use loops
(like for or while) instead of self-calls.
This avoids extra memory used by
function calls.

Algorithm:

- 1) start
- 2) Read n
- 3) set $fact = 1$
- 4) Repeat $fact = fact \times i$ for $i = 1$ to n
- 5) Print $fact$
- 6) stop.

Ex:

```
int fact = 1;
for (int i = 1; i <= n; i++) {
    fact = fact * i;
}
cout << fact;
```

Q. a) Tower of Hanoi: In this problem we

have 3 pegs (A, B, C) & n disks.

The goal is to move all disks from
A (source) to C (destination) using B (auxiliary).

→ NO larger disk can be placed on a
smaller one

→ Only one disk can be moved at a time

recursive logic:

1) Move $n-1$ disks from A → B using C

2) move largest disk from A → C

3) Move $n-1$ disks from B → C using A.

code:

void towerOfHanoi (int n , char from, char aux,

char to) {

if ($n == 1$) {

cout << "move disk 1 from " << from << " to " << to << endl;

return;

}

towerOfHanoi($m-1$, from, to, aux);
cout << "move disk " + n + " from " + from + " to " + to << endl;

3

b) Advantages:

- easy to understand & write
- Reduces code size
- Good for problems that break into subproblems (like tower of Hanoi)

Disadvantages:

- use more memory (stack frames)
- Slower than loops due to repeated calls
- may cause stack overflow for larger inputs.

Set-5:

1) Complexity: means how much space and time an algorithm needs to run.
It helps us understand how efficient an algo is.

time complexity:

Says how much time an algorithm takes to run as input size increases.

Ex: loop from 1 to $n \rightarrow$ takes $O(n)$ time.
nested loops $\rightarrow$ takes $O(n^2)$ time.

Space complexity

Says how much space an algorithm takes during execution.

Ex: simple loop $\rightarrow$ uses $O(1)$ space.

A recursive call $\rightarrow O(n)$ space.

2) Recursion is a process where a function calls itself.

types of recursion:
 $\rightarrow$ direct recursion

→ Tail recursion.

→ Non-tail recursion.

3) Arr[^m_i][ⁿ_j]

Base + $[(i-1) \cdot n + (j-1)] \cdot \text{size}$.

4) Searching means finding the location of an element in a list or array.

There are 2 main techniques:

→ Linear search.

→ Binary search.

Binary search:

→ Works only on sorted arrays.

→ Repeatedly divided into 2 halves.

→ Faster than linear search.

Time complexity: $O(\log n)$

Steps:

1) Find middle element

$\text{mid} = \text{low} + \text{high} / 2$

2) If $\text{key} == \text{arr}[\text{mid}]$ element found

3) If $\text{key} < \text{arr}[\text{mid}]$, search left half

4) If $\text{key} > \text{arr}[\text{mid}]$, search right half

18